

Topic Modeling

What is Topic Modeling?

- A type of **unsupervised machine learning** used for text analysis
- Discovers **latent topics** in a collection of documents without human intervention
- Helps in organizing, summarizing, and understanding large text corpora
- Unlike classification, **topics are not predefined but are derived from patterns in the data**

Key Topic Modeling Techniques

Latent Dirichlet Allocation (LDA)

- The most widely used topic modeling algorithm
- Assumes each document is a mixture of **multiple topics**, and each topic is a mixture of **words**
- Uses Bayesian probability and Dirichlet distribution to infer topic distributions
- Example use case: Organizing scientific articles into research domains (overlapping topics, long text)

Key Topic Modeling Techniques

Non-Negative Matrix Factorization (NMF)

- Uses **matrix factorization** to find hidden patterns in textual data
- Works well for **high-dimensional data** by breaking text down into meaningful components (sparse matrix)
- Does not assume a probabilistic model like LDA (faster)
- Example use case: Extracting themes from customer service chat logs (non-overlapping topics, short text)

Key Topic Modeling Techniques

Latent Semantic Analysis (LSA)

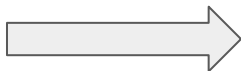
- Applies **Singular Value Decomposition (SVD)** to reduce dimensionality and capture topic-word relationships
- Good for **finding synonym relationships** and **handling noisy data**
- Example use case: Improving search engine results based on contextual meaning

LDA Overview

- **Probabilistic generative model** used for discovering topics in a collection of documents
- Each document is a **mixture of multiple topics**, and each topic is a **distribution over words**
- Utilizes **Dirichlet priors** to control topic distributions
- Uses **Bayesian inference techniques** such as **Gibbs sampling** or **Variational Inference**
- Requires specifying the number of topics (K) beforehand

How LDA Works (High-Level)

我



我是誰：

品種 → 電動人 (topics)

興趣 → 漫畫、電動

人生地不熟，哪裡適合 電動人去呢？

Collect Documents

地區 (Documents)

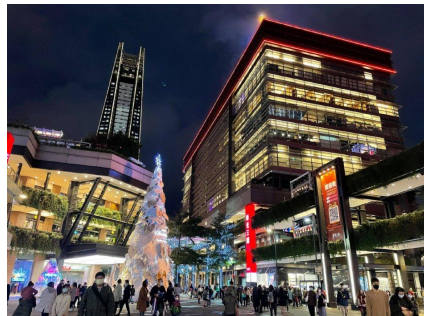


朋友 (Words)



初始狀態 (隨便猜)

地區 (Documents)



地區特性 / 朋友興趣
(Topics)

大雄是文青

靜香是網美

朋友 (Words)



Iterative Update

地區 (Documents)



地區特性 / 朋友興趣
(Topics)



朋友 (Words)



Iterative Update

地區 (Documents)



再仔細觀察其他人...

地區特性 / 朋友興趣
(Topics)

大雄是文青

朋友 (Words)



電動人



電動人



電動人

Iterative Update

地區 (Documents)



地區特性 / 朋友興趣
(Topics)

大雄是電動人

朋友 (Words)



電動人



電動人



電動人

Iterative Update

地區 (Documents)



經過多次迭代後，發現光華商場裡
出現許多電動人，因此定義出：光
華商場 = 90% 電動人主題

地區特性 / 朋友興趣
(Topics)

大雄是電動人

朋友 (Words)



電動人



電動人



電動人

Iterative Update

地區 (Documents)



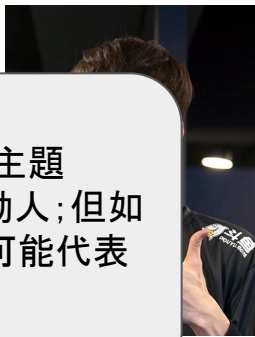
地區特性 / 朋友興趣
(Topics)

大雄是電動人

朋友 (Words)



Note: 同一個朋友(word)可以出現在不同主題
例如:Faker出現在光華商場, 他代表的是 電動人; 但如果Faker出現在信義區百貨公司代言手錶, 他可能代表 時尚/商業



電動人



電動人



電動人

Iterative Update

地區 (Documents)



地區特性 / 朋友興趣
(Topics)

大雄在光華商場的
標籤為電動人



電動人



電動人

朋友 (Words)



電動人

Iterative Update

地區 (Documents)



地區特性 / 朋友興趣
(Topics)

大雄在光華商場的
標籤為電動人

朋友 (Words)



如果我們在一个沒去過的地下街 (new document), 看到一堆Faker 和研磨在走動, 雖然我們不知道那是什麼地方, 但根據LDA可以推測那邊的主題



電動人



電動人



電動人

同樣地，經過多次迭代我們可以推論信義區的主題分佈為...

50%: 購物仔



15%: 網美



30%: 電影仔



5%: 電動人



Key Concepts in LDA

- **Documents as Topic Mixtures**

- Example: A research paper might be 60% about AI and 40% about statistics

- **Topics as Word Distributions**

- Example:
 - Topic 1 (Sports): basketball (0.2), football (0.15), tennis (0.1), player (0.05)...
 - Topic 2 (Technology): AI (0.2), machine (0.15), learning (0.1), neural (0.05)...

- **Dirichlet Distribution**

- Alpha (α): Controls how documents mix topics
 - High $\alpha \rightarrow$ Documents contain many topics evenly
 - Low $\alpha \rightarrow$ Documents are dominated by fewer topics
- Beta (β): Controls how topics mix words
 - High $\beta \rightarrow$ Topics share more words
 - Low $\beta \rightarrow$ Topics are more distinct

Step-by-Step Process

1. **Initialize topics randomly** for words in each document (Dirichlet Distribution).
2. Iterative topic assignment using Gibbs Sampling
 - a. Reassign words to topics based on:
 - i. How frequently a topic appears in a document
 - ii. How frequently a word appears in a topic across **all documents**
3. **Repeat until convergence** to stabilize topic distributions.
4. Output final results:
 - a. Topic-word distributions (which words belong to which topics)
 - b. Document-topic distributions (which topics dominate which documents)

Gibbs Sampling

For each word w_i in document d_i , reassign its topic based on the **conditional probability**:

$$P(z_i = k | w_i, d_i) \propto \frac{n_{dk} + \alpha}{\sum_k n_{dk} + K\alpha} \times \frac{n_{kw} + \beta}{\sum_w n_{kw} + V\beta}$$

where:

- n_{dk} = Number of words in document d assigned to topic k .
- n_{kw} = Number of times word w appears in topic k .
- α = Dirichlet prior for **document-topic distribution**.
- β = Dirichlet prior for **topic-word distribution**.
- K = Total number of topics.
- V = Total vocabulary size.

Gibbs Sampling

For each word w_i in document d_i , reassign its topic based on the **conditional probability**:

$$P(z_i = k | w_i, d_i) \propto \frac{n_{dk} + \alpha}{\sum_k n_{dk} + K\alpha} \times \frac{n_{kw} + \beta}{\sum_w n_{kw} + V\beta}$$

where:

P(標籤 = 網美 | 其他人的狀態)

- n_{dk} = Number of words in document d assigned to topic k .
- n_{kw} = Number of times word w appears in topic k .
- α = Dirichlet prior for **document-topic distribution**.
- β = Dirichlet prior for **topic-word distribution**.
- K = Total number of topics.
- V = Total vocabulary size.

Gibbs Sampling

For each word w_i in document d_i , reassign its topic based on the **conditional probability**:

$$P(z_i = k | w_i, d_i) \propto \frac{n_{dk} + \alpha}{\sum_k n_{dk} + K\alpha} \times \frac{n_{kw} + \beta}{\sum_w n_{kw} + V\beta}$$

where:

該地區目前的網美比例 + α

- n_{dk} = Number of words in document d assigned to topic k .
- n_{kw} = Number of times word w appears in topic k .
- α = Dirichlet prior for **document-topic distribution**.
- β = Dirichlet prior for **topic-word distribution**.
- K = Total number of topics.
- V = Total vocabulary size.

Gibbs Sampling

For each word w_i in document d_i , reassign its topic based on the **conditional probability**:

$$P(z_i = k | w_i, d_i) \propto \frac{n_{dk} + \alpha}{\sum_k n_{dk} + K\alpha} \times \frac{n_{kw} + \beta}{\sum_w n_{kw} + V\beta}$$

where:

全台北靜香被歸類為網美的次數 + $\square$

- n_{dk} = Number of words in document d assigned to topic k .
- n_{kw} = Number of times word w appears in topic k .
- α = Dirichlet prior for **document-topic distribution**.
- β = Dirichlet prior for **topic-word distribution**.
- K = Total number of topics.
- V = Total vocabulary size.

API Perspective: Inputs and Outputs

- Inputs: count vectors (bag of words)

[illegible]

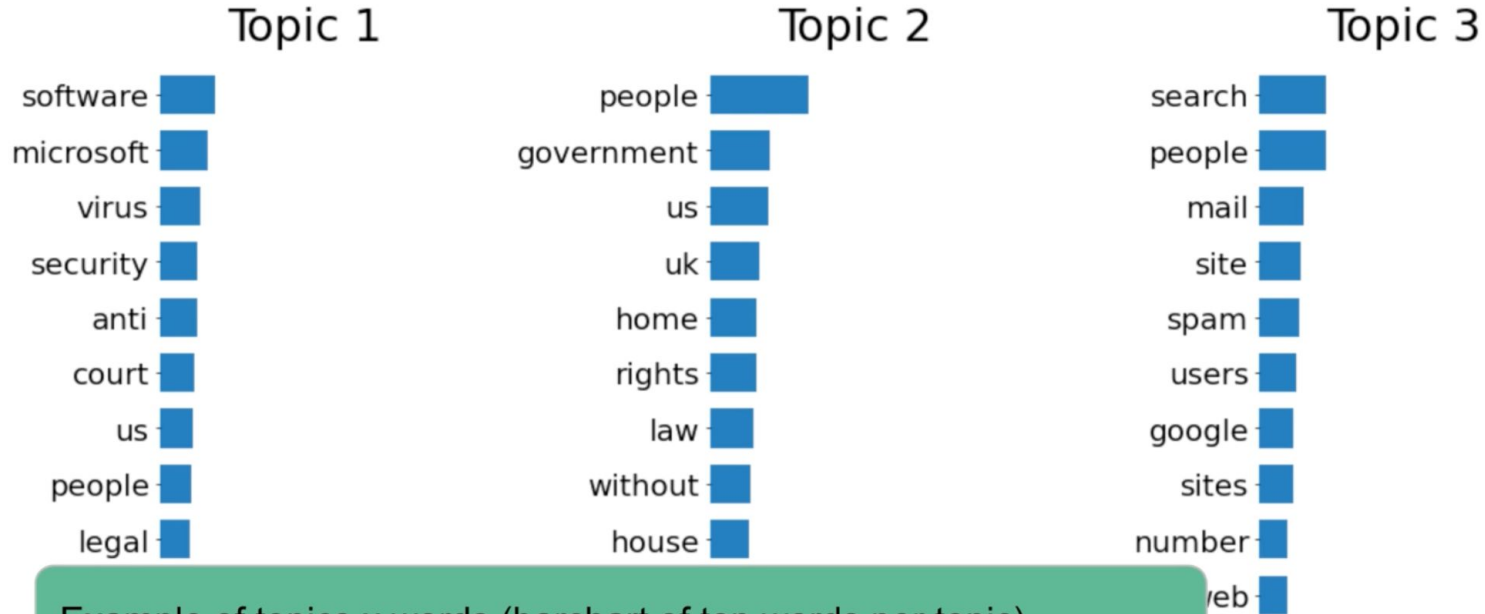
API Perspective: Inputs and Outputs

- 2 matrixes as output: **topics x words**, document x topics

	word1	word2	word3	word4	word5	word6	word7	word8
topic1								
topic2								
topic3								
topic4								
topic5								

API Perspective: Inputs and Outputs

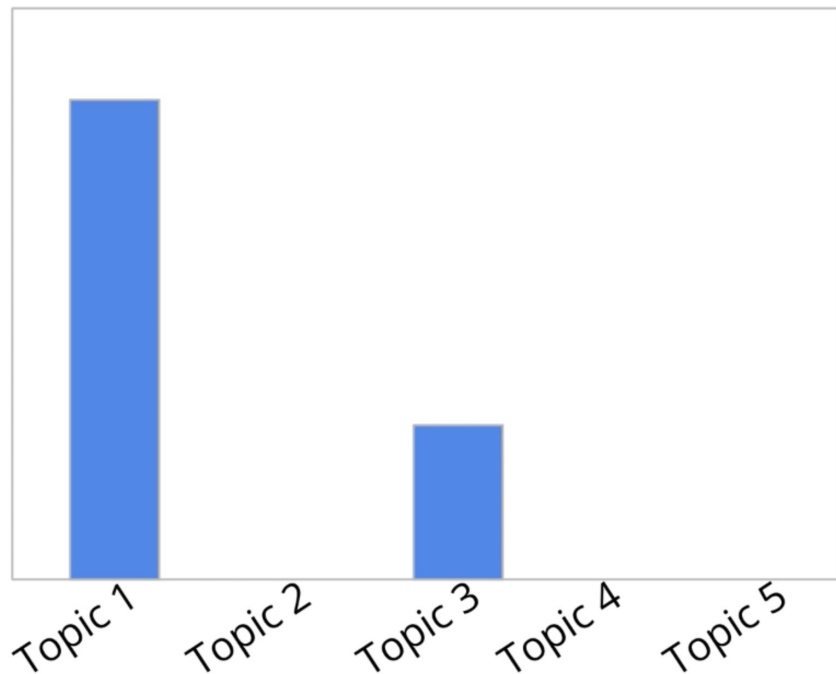
- 2 matrixes as output: **topics x words**, document x topics



Example of topics x words (barchart of top words per topic)

API Perspective: Inputs and Outputs

- 2 matrixes as output: topics x words, **document x topics**



API Perspective: Inputs and Outputs

- 2 matrixes as output: topics x words, **document x topics**



The Actual API

```
X = CountVectorizer().fit_transform(text)

lda = LatentDirichletAllocation()
lda.fit(X) # not fit(X, Y)
Z = lda.transform(X) # returns docs x topics matrix
```

The Actual API

```
Z = lda.fit_transform(X) # all at once
```

```
# where is the topics x words matrix?
```

```
topics = lda.components_
```

```
# the topics are "internal" to the model, documents are not
```

```
# we can pass in new data to be transformed under the
```

```
# existing topics
```

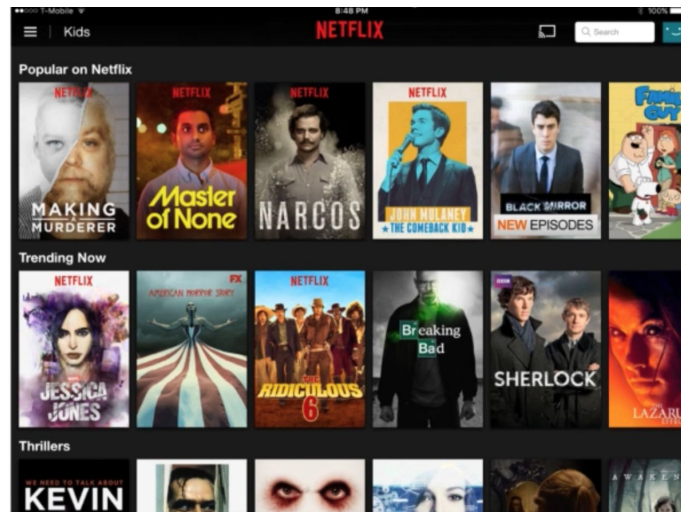
```
Z_test = lda.transform(X_test)
```

Non-Negative Matrix Factorization (NMF)

- Factorization-based topic modeling method.
- Decomposes a **document-term** matrix into two lower-dimensional matrices.
- Ensures that all matrix values remain **non-negative**, making the results easier to interpret.
- Works well with TF-IDF-weighted text representations.
- Requires predefining the number of topics (K) beforehand.

Recommender Systems

- You want to recommend movies to users, but there many ways
- Let's simplify this: a good recommendation is a movie that a user would rate highly
- Recommendation list is simply a list of movies ordered by predicted rating
- Our task is simple: predict unknown ratings



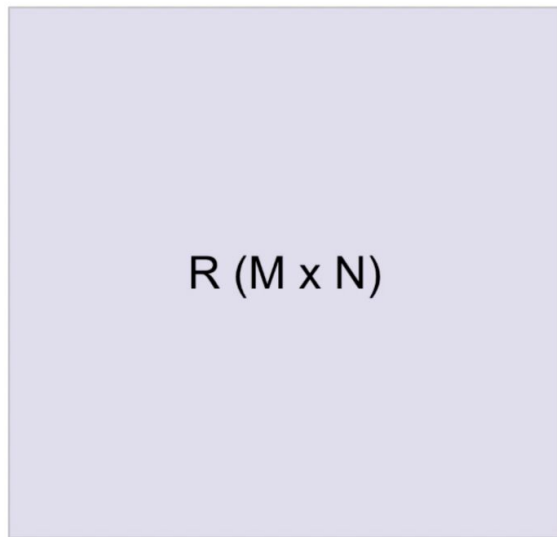
The Rating Matrix

- M users (rows), N movies (columns) $\rightarrow$ M x N matrix
- Important: we don't know most of the values
- Why? Just think about how many movies exist. How many have you seen?
Rated?
- It's a **sparse** matrix
- Different notion of sparsity from TF-IDF (zero vs. missing)
- Goal: fill in missing values

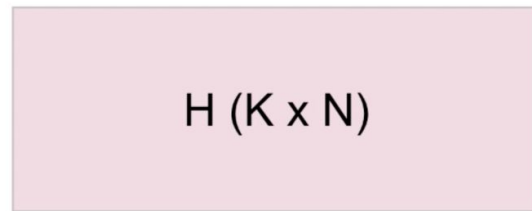
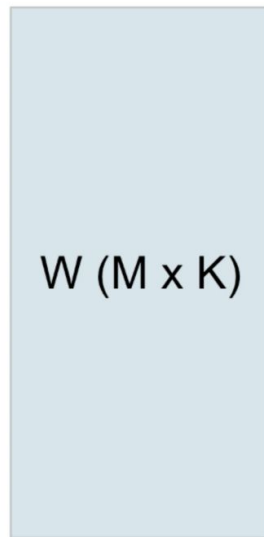
						
	5		8	7		
	9	10	8	9		
	6	6		6	10	7
	8				6	10

Matrix Factorization

$$R \approx WH$$



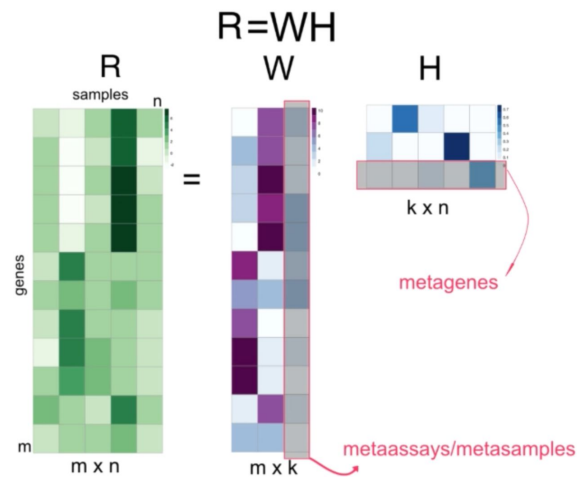
$\approx$



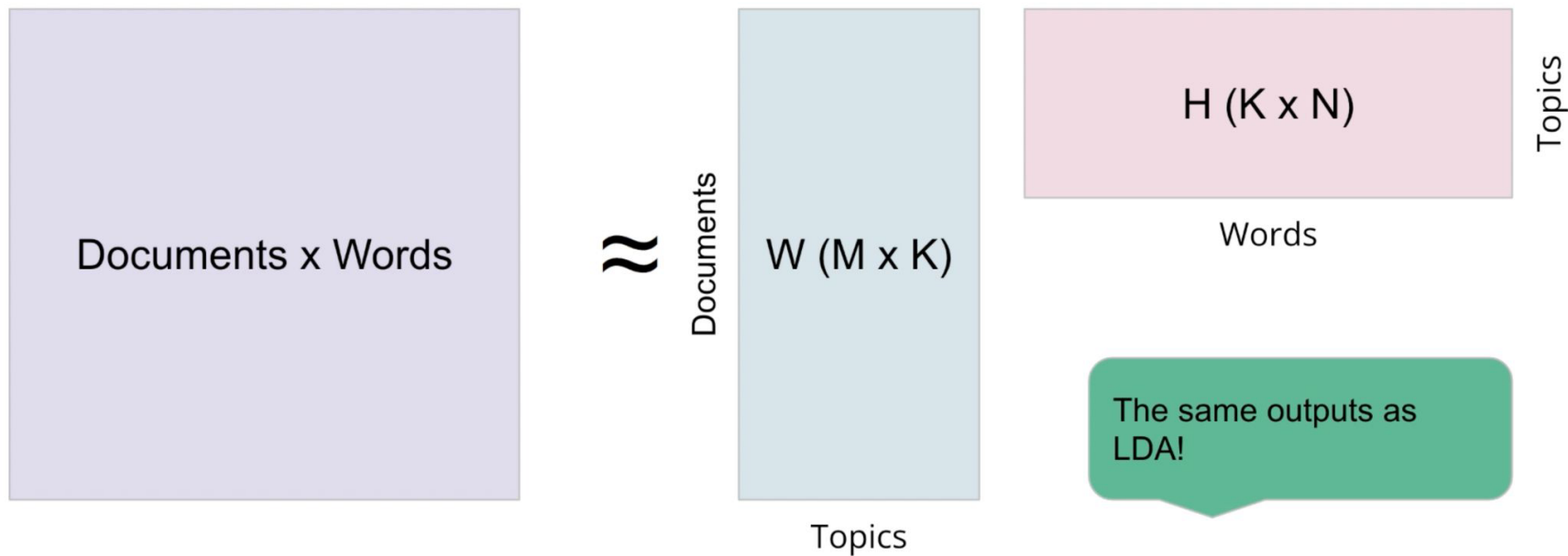
When we multiply W and H together, the result is an $M \times N$ matrix, with **no missing values!**

How to Find W & H?

- We “train” ML models by defining a loss function and then finding the values of W and H that would minimize that loss
- E.g. squared error between predictions and true ratings



NMF Output



Latent Semantic Analysis

- LSA is just the application of a well-known ML technique (**SVD**) applied to a term-document matrix
- Just like applying Naive Bayes or Logistic Regression to vectors
- By studying these techniques, you'll improve your skills in other areas too (e.g. computer vision)

Singular Value Decomposition (SVD) Intuition

- SVD is useful for reducing dimensionality
- SVD is useful for visualizing your data
- SVD works by finding the best rotation of your data points

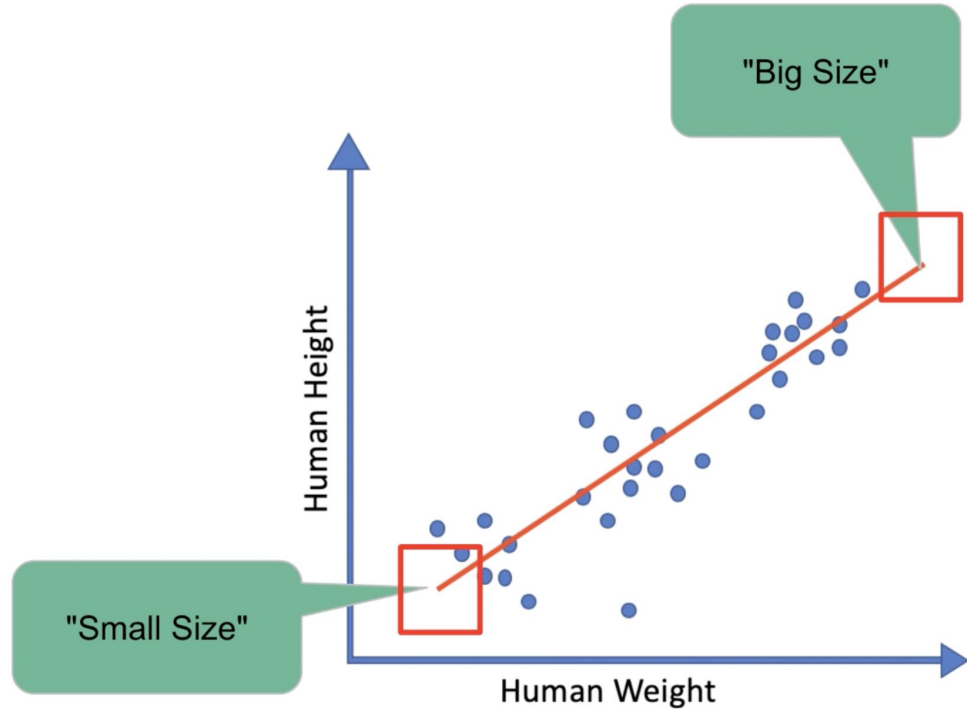
Reducing Dimensionality

- In NLP, data is typically high-dimensional (many columns)
- Often, # features >> # samples
- More dimensions = more data to process = more time spent
- Reduce dimensionality = pipeline is more efficient and fast

	I	love	dogs	hate	and	knitting	is	my	hobby	passion
Doc 1	0.18	0.48	0.18							
Doc 2	0.18		0.18	0.48	0.18	0.18				
Doc 3					0.18	0.18	0.48	0.95	0.48	0.48

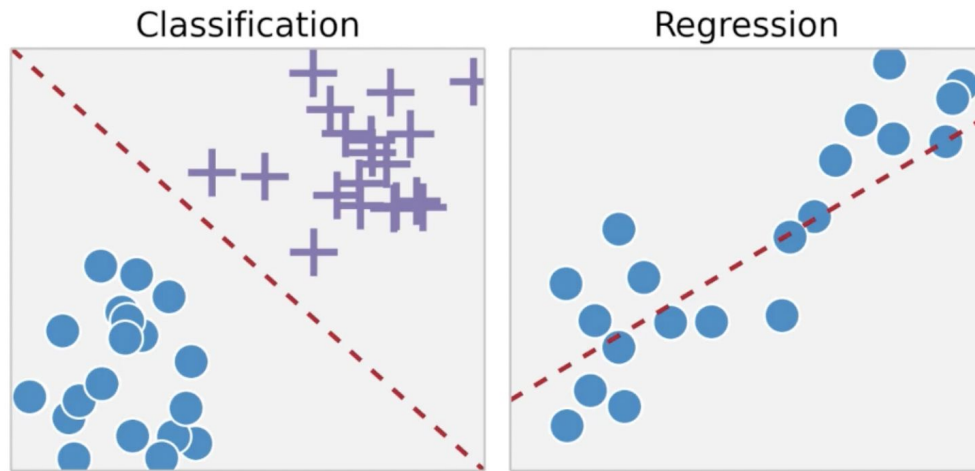
Example: Reducing Dimensionality

- Project all the data points down to the line
- The data is described by a single variable, **size**
- We only need 1 dimension



Visualization

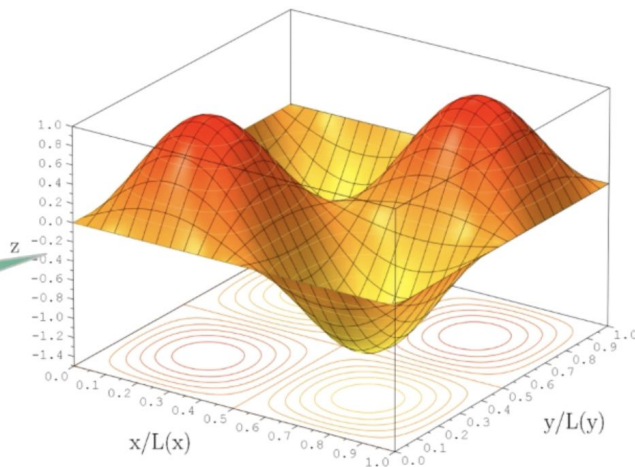
- Visualization is always useful in data science and ML
- The world is only 3-D, how can we see data that is 1000-D?



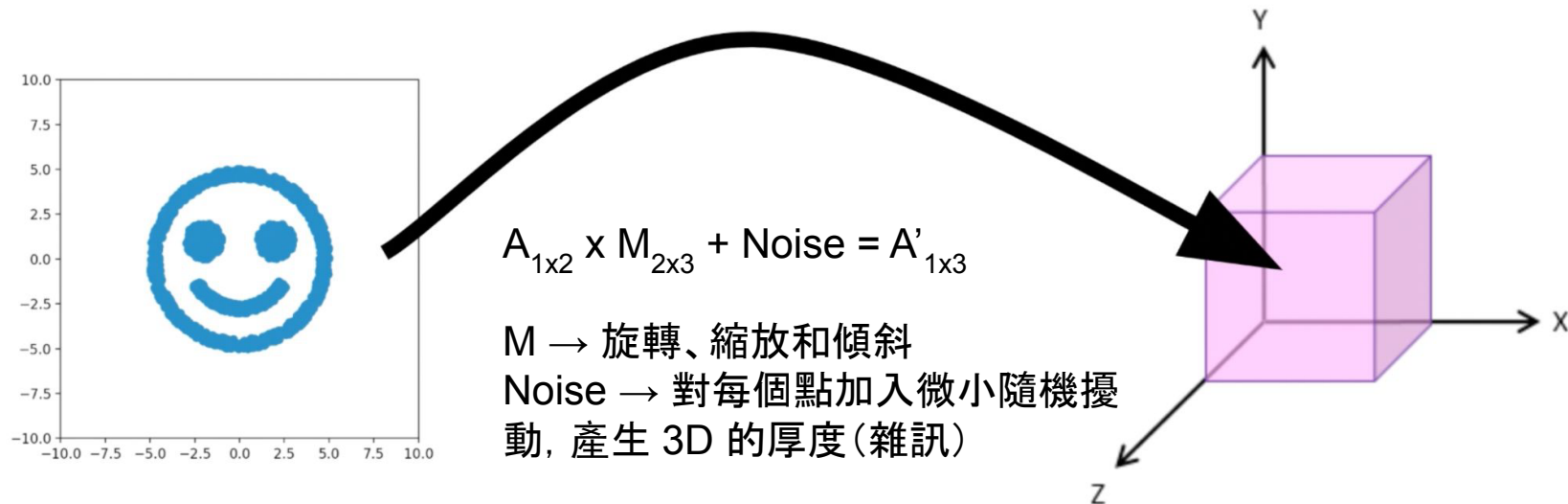
Rotation

- To understand how this works, we should think “backwards”
- Why do we do dimension reduction? We believe our data has smaller number of dimensions, but was embedded into a larger dimensional space

A 2-D object embedded
in 3-D space

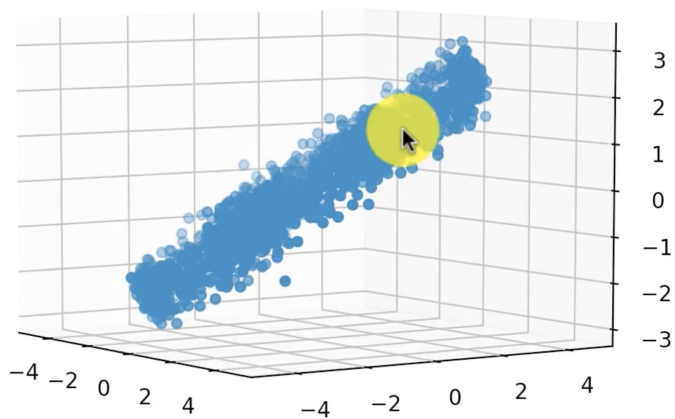
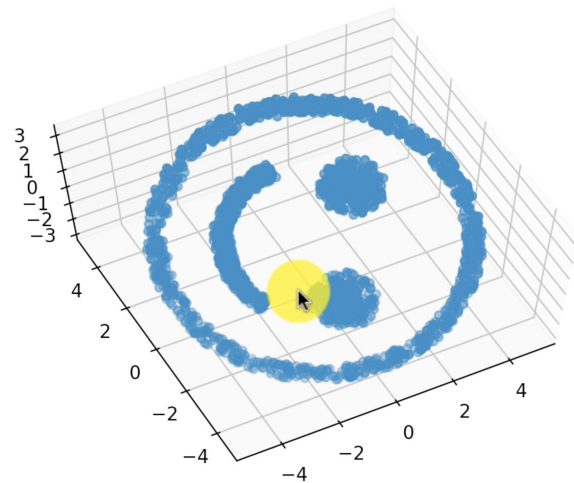
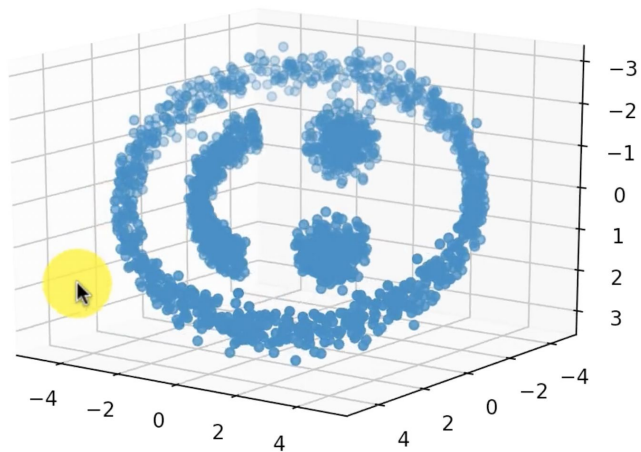


Embed the 2-D Object in 3-D Space



$$A_{1 \times 2} = [x, y]$$

Different Angles



Thickness is the noise
we add

After Applying SVD

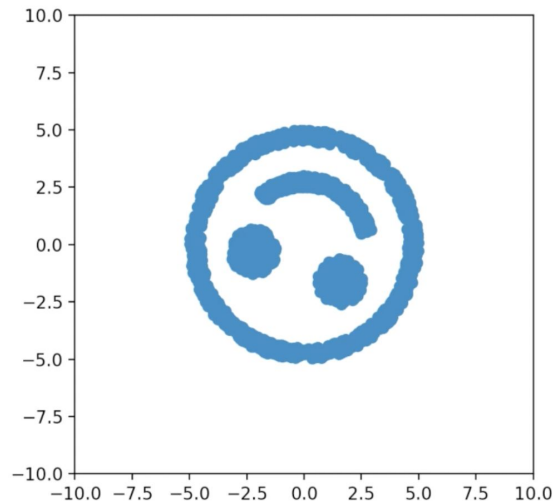
- We recover the happy face
- It's not at the right orientation, but that's ok

$$A = U\Sigma V^T$$

V^T : 找出資料的最佳觀測視角(旋轉空間)

Σ : 決定每個視角的重要性(大的值是特徵, 小的值是雜訊)

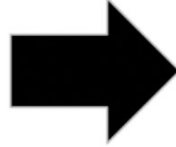
丟棄 Σ 中的微小數值後, 重新計算出的 A_{clean}
就是無雜訊的笑臉



SVD found the right orientation, and threw away the 3rd "noise" dimension

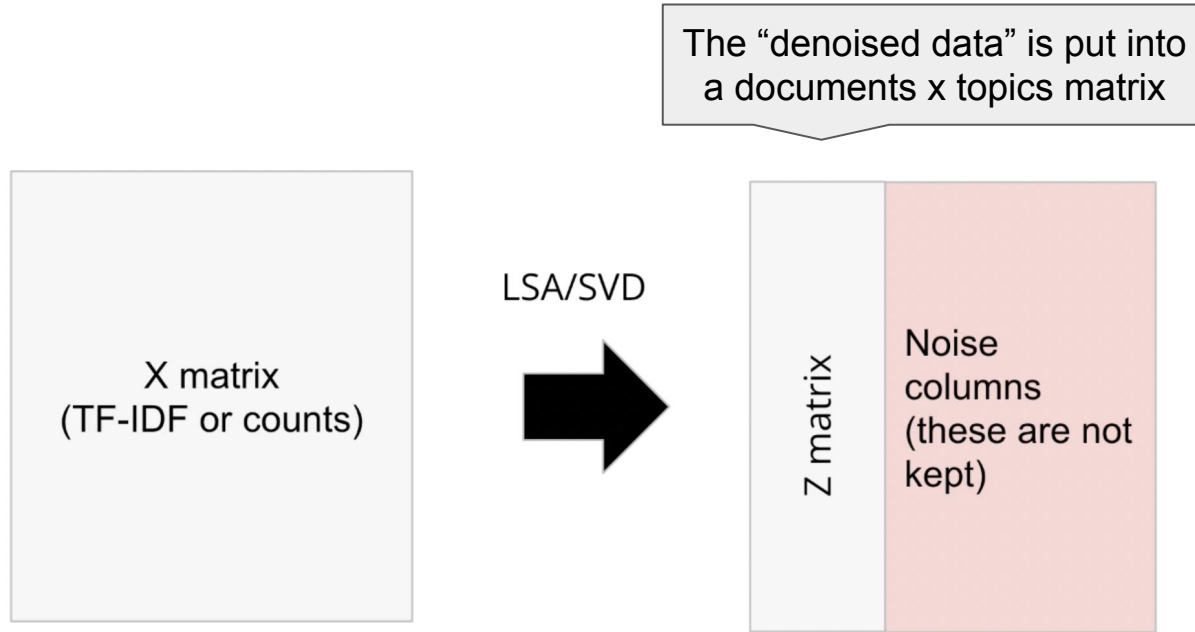
Noise Removal

Original data: each dimension is a bit of data + noise

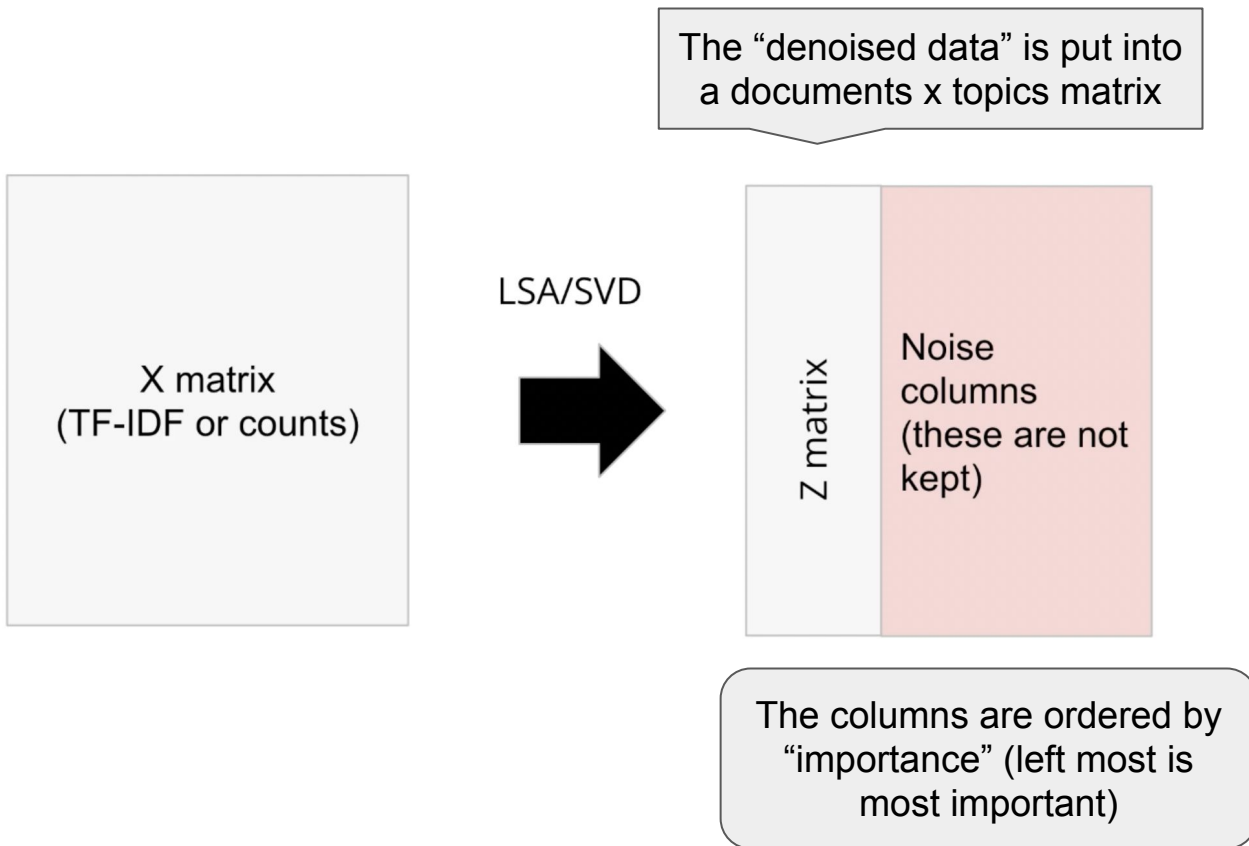


Rotated data: axes are aligned so that each dimension is pure data or pure noise

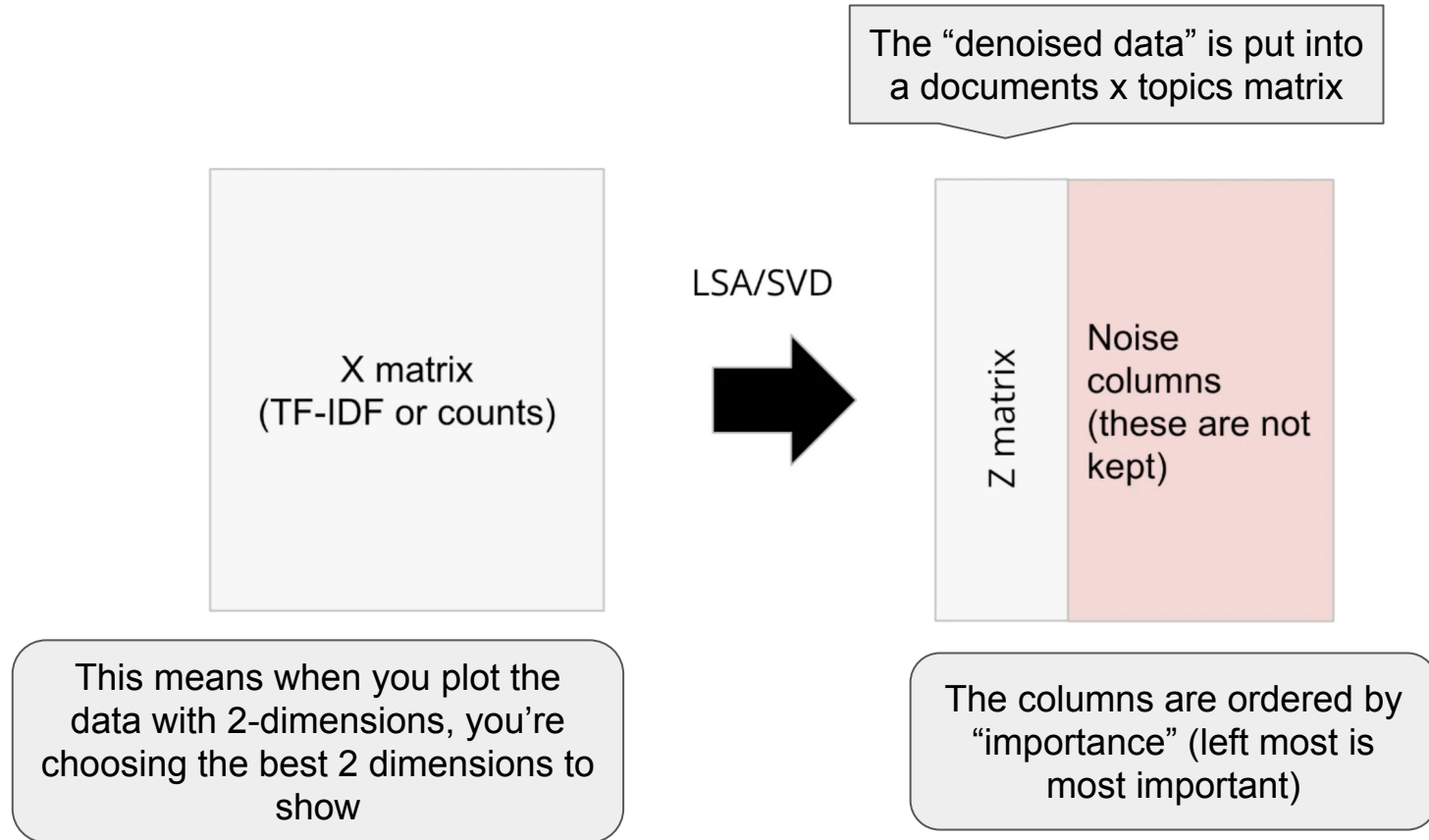
What Does LSA Do?



What Does LSA Do?



What Does LSA Do?



LSA in Code

```
# can use Scikit-Learn or Numpy/Scipy
# Numpy/Scipy is more useful for detailed computations
from sklearn.decomposition import TruncatedSVD

# create an object - you use num components
# e.g. 2 for visualization, cross-validation, etc.
model = TruncatedSVD(n_components=2)

# train/fit the model
model.fit(X)

# alternatively, can use term-document matrix
# if you want word vectors instead of document vectors
model.fit(X.T)
```

LSA in Code

```
# transform data
```

```
Z = model.transform(X)
```

```
# all in one step
```

```
Z = model.fit_transform(X)
```

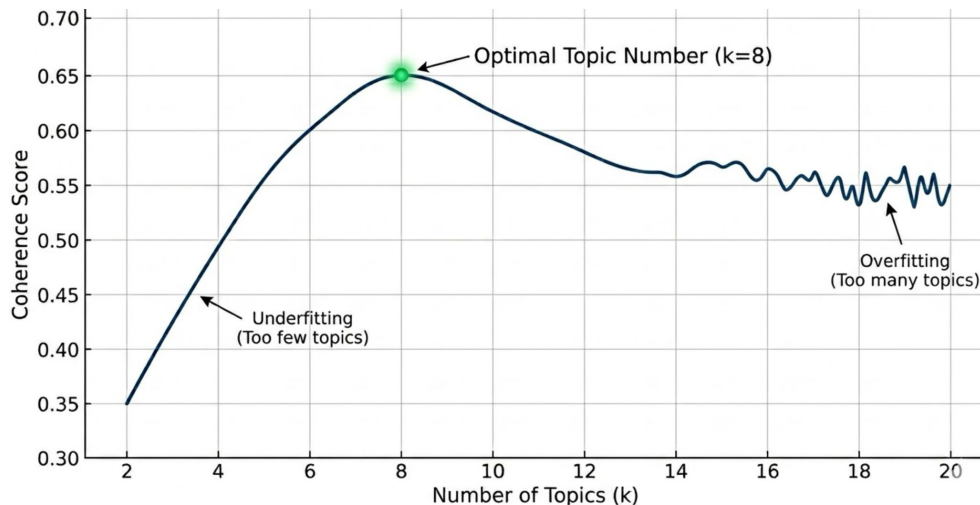
```
# note: Z has shape N x K (K = number of "hidden topics")
```

```
# note: unsupervised learning
```

Choosing the Right Number of Topics

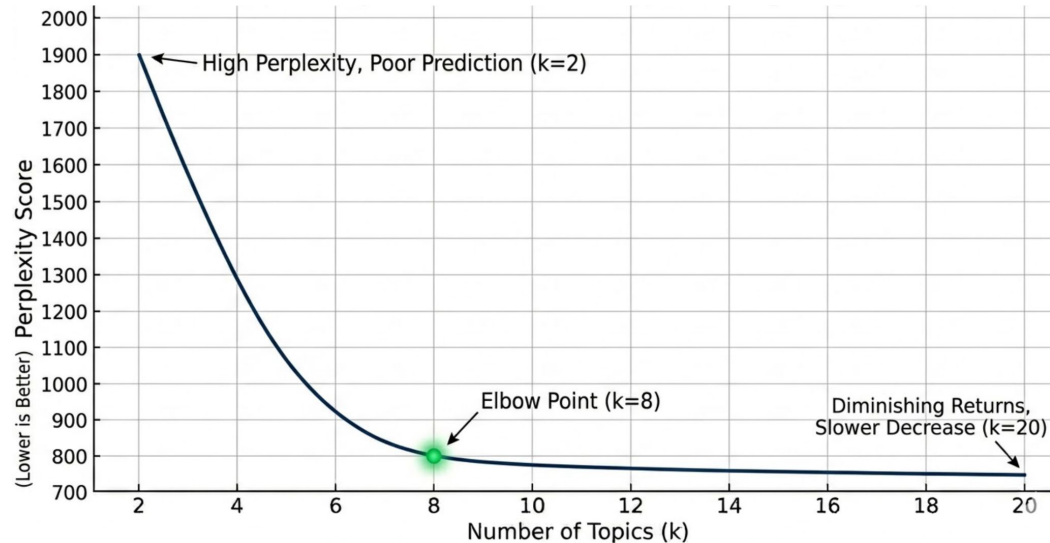
- Setting **too few** topics: Oversimplifies the data, leading to broad and uninformative topics
- Setting **too many** topics: Overfitting, making results harder to interpret
- Methods to determine optimal topic numbers:
 - **Coherence Score**
 - Measures how semantically related the top topic words are
 - Higher = more meaningful topics
 - **Perplexity Score**
 - Measures how well the model predicts unseen data
 - Lower = better generalization
 - Mainly for LDA

Coherence Score



- Are top words in a topic meaningfully related?
- Example:
 - High coherence: apple, banana, grape, orange
 - Low coherence: apple, engine, democracy, pillow
- Computed using **word co-occurrence** or **semantic similarity** (e.g., C_v, UMass)
- **Higher is better**
- Good for evaluating topic **interpretability**

Perplexity Score



- Measures model's ability to predict **unseen documents**
- Lower perplexity → better generalization
- Common in language modeling
- But:
 - Doesn't guarantee meaningful or interpretable topics
 - A model with low perplexity can still produce incoherent topics
- Not always aligned with human judgment

Calculating Perplexity Score (skip)

對於一個語料庫 D :

$$\text{Perplexity}(D) = \exp \left(-\frac{\sum_{d \in D} \log P(w_d)}{\sum_{d \in D} N_d} \right)$$

- $P(w_d)$: 模型對文件 d 的所有詞語的聯合機率
- N_d : 文件 d 中的總詞數
- 結果越低 $\rightarrow$ 表示模型能更好地解釋這些資料

Challenges in Topic Modeling

- Choosing the right number of topics
- Interpreting abstract topics
 - Topics are probabilistic, not always clear-cut
- Handling short texts
 - Tweets, chat messages, and fragmented text pose challenges
- Noise in text
 - Presence of irrelevant words and misspellings affects quality